EMBEDDED SYSTEM DESIGN

# 嵌入式系统设计
# ——基于Cortex-M处理器与RTOS构建

## 实验 05 RTOS任务间通信实验

杭州電子科技大学
HANGZHOU DIANZI UNIVERSITY

# 多任务系统基本概念——为什么使用信号量与消息队列

在RTOS应用软件设计中，使用信号量或消息队列相比全局变量有以下几个优点。

- 使用全局变量会增加耦合度，降低内聚性，不符合软件设计思想。
- 使用全局变量影响封装性、移植性和可读性，使用过多的全局变量会严重降低代码可维护性和稳定性。
- 信号量和消息队列等可以将任务阻塞，避免抢占CPU资源。

# 多任务系统基本概念——消息队列

CMSIS常见的任务间通信方式:

- 消息队列（Message Queues）
- 信号量（Semaphores）
- 互斥锁（Mutexes）
- 任务通知（Notifications）

这几种通信方式中，除了任务通知，其他几种方式都是基于消息队列来实现的。

# 多任务系统基本概念——消息队列

- 消息队列作为任务间通信的一种数据结构，支持在任务与任务间、中断和任务间传递消息内容。

- 消息队列的数据结构实现： FIFO（先入先出）缓冲区，即最先写入的消息也是最先被读出的。

- 消息队列可以存储有限个具有确定长度的数据单元，可以保存的最大单元数目被称为队列的“深度”，在消息队列创建时需要设定其深度和每个单元的大小。

# 多任务系统基本概念——消息队列

- 当某个任务试图从一个消息队列读取数据时，可以指定一个阻塞超时时间。
- 在这段时间内，如果消息队列为空，该任务将保持阻塞状态以等待新的消息。
- 当其它任务或中断向其等待的消息队列写入了数据，该任务将自动从阻塞状态转入就绪状态。

# 多任务系统基本概念——消息队列

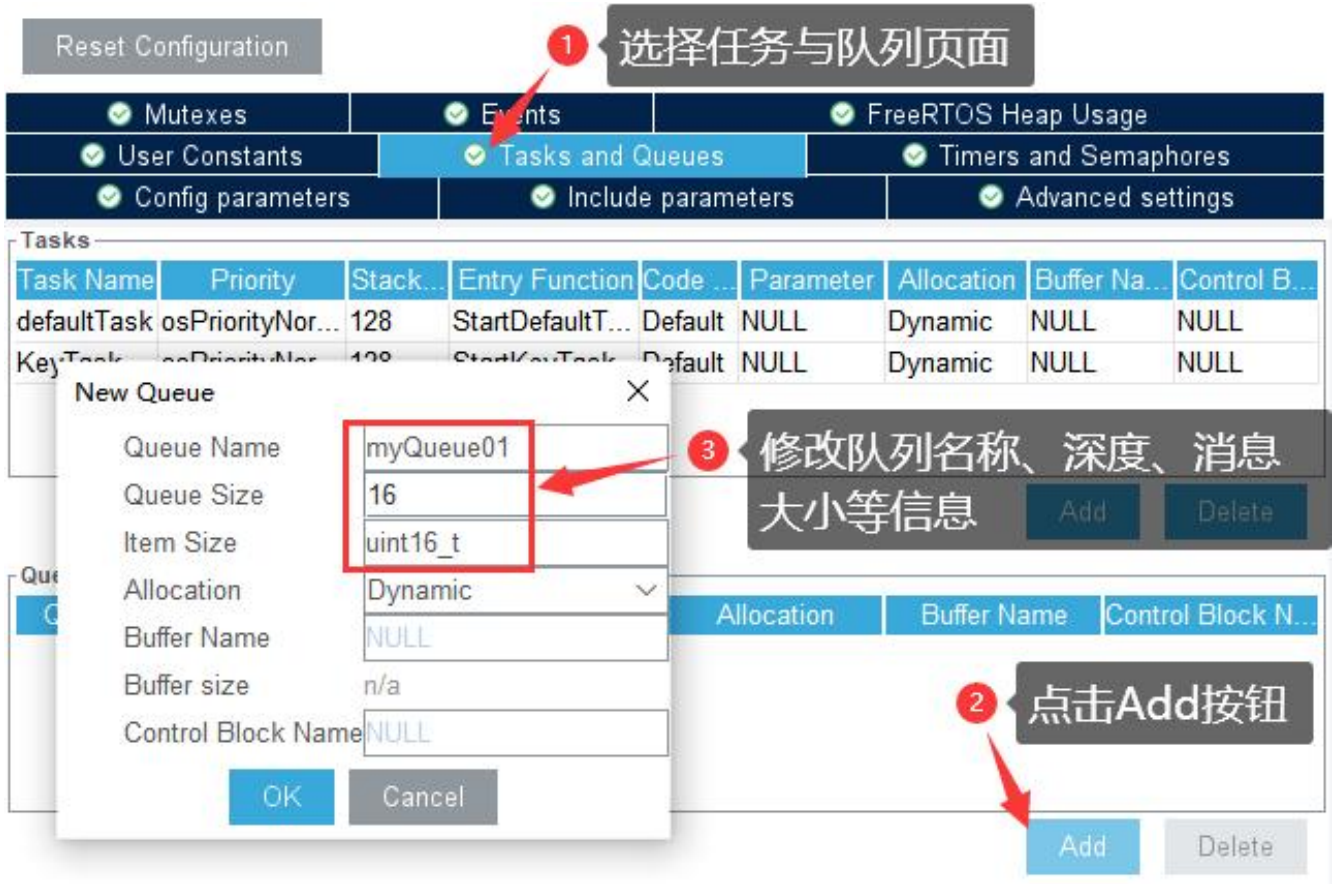
FreeRTOS中关于消息队列的常用函数如下表所示。

| 功能分类 | FreeRTOS原版函数 | CMSIS v2封装函数 | 说明 |
|---|---|---|---|
| 消息队列 | xQueueCreate | **osMessageQueueNew** | 创建并初始化消息队列对象 |
| | xQueueSendToBack<br>xQueueSendToBackFromISR | **osMessageQueuePut** | 将消息放入队列，如果队列已满则一直等待到超时 |
| | xQueueReceive<br>xQueueReceiveFromISR | **osMessageQueueGet** | 从队列获取消息，如果队列为空则一直等待到超时 |
| | uxQueueMessagesWaiting | osMessageQueueGetCount | 获取消息队列中排队的消息数量 |
| | uxQueueSpacesAvailable | osMessageQueueGetSpace | 获取消息队列的可用插槽数量 |
| | xQueueReset | osMessageQueueReset | 将消息队列重置为初始空状态 |
| | vQueueDelete | osMessageQueueDelete | 删除一个消息队列对象 |

# 多任务系统基本概念——消息队列

可以在CubeMX的FreeRTOS设置页面中，给工程先配置好要用到的消息队列

这样导出MDK工程代码中就能在任务间使用Put与Get两个函数进行消息传递与读取操作了。配置如图所示：



# 多任务系统基本概念——消息队列

CubeMX导出的MDK工程中，会在freertos.c文件中自动生成消息队列变量，并在MX_FREERTOS_Init函数中自动创建之前配置的消息队列，消息队列应用的示例代码如下：

```
// 定义消息队列类型全局变量
osMessageQueueId_t myQueue01Handle;
const osMessageQueueAttr_t myQueue01_attributes = {
  .name = "myQueue01"
};
// 在MX_FREERTOS_Init函数或其它任务中创建消息队列
myQueue01Handle = osMessageQueueNew (16,
            sizeof(uint16_t), &myQueue01_attributes);
```

```
// 在按键扫描任务扫描键码并写入消息队列
void StartKeyTask(void *argument) {
  for(;;) {
    ...
    // 将扫描到的键码key值写入消息队列
    osMessageQueuePut(myQueue01Handle, &key, 0, 0);
    ...
  }
}
```

```
// 在默认任务中获取消息队列中的按键消息
void StartDefaultTask(void *argument) {
  for(;;) {
    uint16_t key = 0;
    // 读取按键消息到变量key，读取时最多等待10ms
    if (osMessageQueueGet(myQueue01Handle,
        &key, 0, 10) == osOK) {
      // 获取按键消息成功
      ...
    }
    else {
      // 没有获取到按键消息
      ...
    }
  }
}
```

# 多任务系统基本概念——信号量

信号量是操作系统中的一个重要概念，其一般用来进行资源管理和任务同步。

FreeRTOS中的信号量：

- 二值信号量
- 计数型信号量
- 互斥信号量
- 递归互斥信号量

# 多任务系统基本概念——信号量

资源管理:

- 信号量用于控制共享资源访问的场景相当于一个上锁机制
- 程序代码只有获得了这个锁的钥匙才能够执行。
- 当共享资源的数量唯一，其状态就只有两个(使用或未使用)
- 这两个状态就可用一个二值信号量来表示：当共享资源的数量大于1，其状态值就可用一个计数型信号量表示。

# 多任务系统基本概念——信号量

任务同步:

- 用于任务与任务之间或中断与任务之间的同步
- 在执行中断服务函数时可以通过向任务发送信号量来通知任务所期待的事件发生了
- 退出中断服务函数以后，在系统内核的调度下同步的任务就会执行

# 多任务系统基本概念——信号量

编写小型项目代码时使用全局变量没有太大问题，因为全局变量的同步操作简单、内存地址固定、读写效率高。

随着项目规模加大和逻辑复杂度增加，RTOS的资源管理和任务同步更推荐使用信号量。

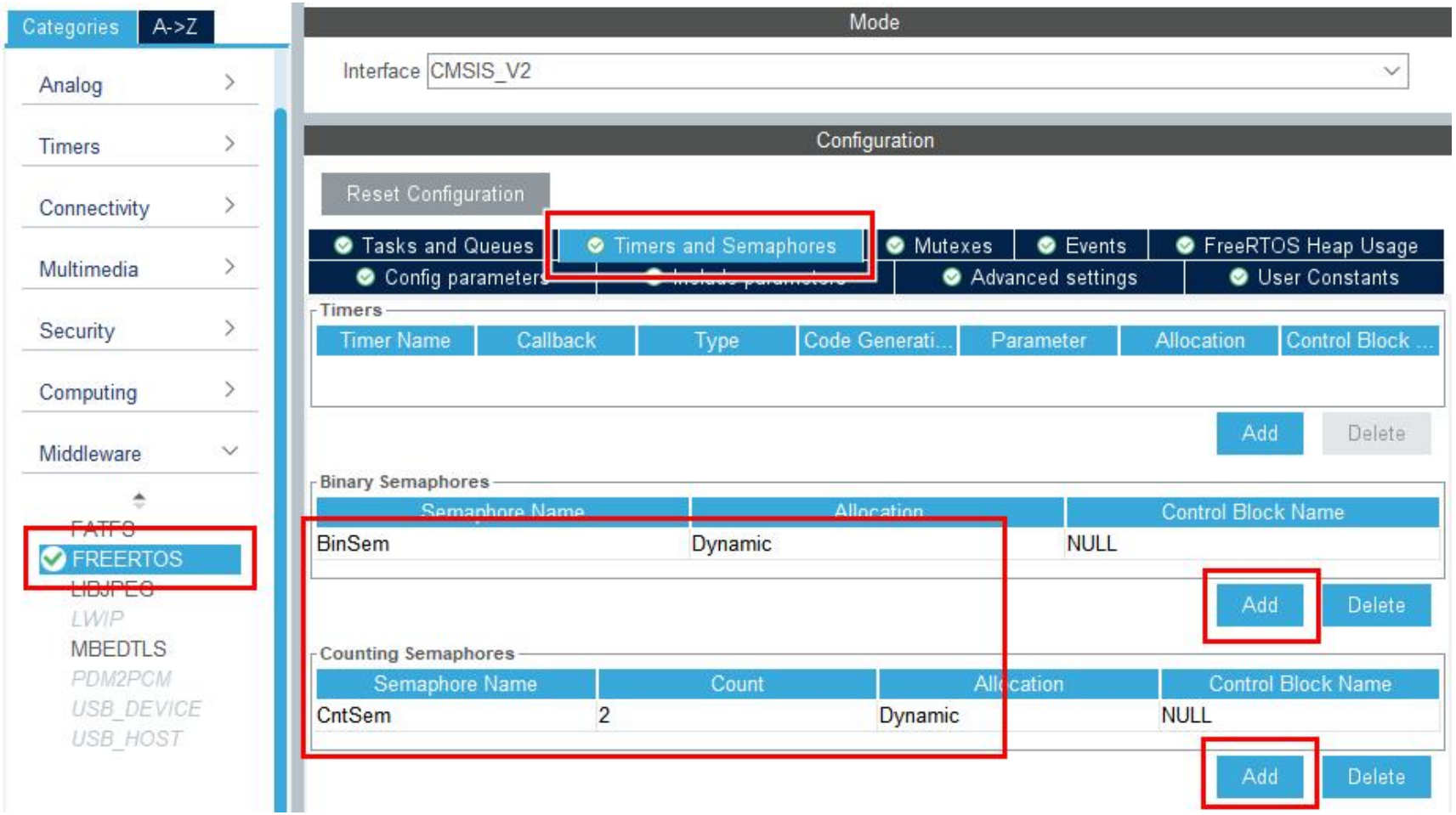
# 多任务系统基本概念——信号量

FreeRTOS中关于信号量的常用函数如下表所示。

| 功能分类 | FreeRTOS原版函数 | CMSIS v2封装函数 | 说明 |
|---|---|---|---|
| 信号量 | xSemaphoreCreateBinary<br>xSemaphoreCreateCounting | **osSemaphoreNew** | 创建并初始化信号量对象 |
| | xSemaphoreTake<br>xSemaphoreTakeFromISR | **osSemaphoreAcquire** | 如果信号量令牌可用(计数大于0)，立即返回并减少令牌计数，否则一直等待到超时 |
| | xSemaphoreGive<br>xSemaphoreGiveFromISR | **osSemaphoreRelease** | 释放一个信号量令牌(计数加1)直到最大计数值 |
| | uxSemaphoreGetCount | osSemaphoreGetCount | 获取当前的信号量令牌计数 |
| | vSemaphoreDelete | osSemaphoreDelete | 删除一个信号量对象 |

# 多任务系统基本概念——信号量

如下图所示，用户也可以在CubeMX的FreeRTOS设置页面中给工程先配置好要用到的信号量。



# 多任务系统基本概念——互斥量

互斥量是二进制信号量的一个变种。

互斥量和信号量的主要区别：互斥量具有优先级继承机制，信号量没有。

# 多任务系统基本概念——互斥量

如果某个受到一个互斥量保护的临界资源正在被一个低优先级任务使用:

- 此时的互斥量是闭锁状态，即没有任务能申请到这个互斥量;

如果此时一个高优先级任务想要对这个资源进行访问，去申请这个互斥量:

- 高优先级任务会因为申请不到互斥量而进入阻塞态
- 持有该互斥量的任务的优先级将临时提升到与高优先级任务的优先级相同，这个优先级提升的过程叫作优先级继承。

# 多任务系统基本概念——互斥量

“优先级翻转”:

1、把互斥量换为信号量
2、高优先级任务因为想要访问资源但申请不到信号量进入阻塞态
3、系统并没有将持有该信号量的低优先级任务临时提升到高优先级

可能其他中优先级的任务会抢占低优先级任务，导致高优先级任务的阻塞状态延长。

互斥量的优先级继承机制能确保高优先级任务进入阻塞状态的时间尽可能短，以及将已经出现的“优先级翻转”危害降低到最小。

# 多任务系统基本概念——互斥量

实现临界资源的多重保护

还有一种互斥量被称为递归互斥量，它和普通互斥量的区别在于:

- 递归互斥量在获取了一次互斥锁之后，没有释放互斥锁之前还可以再次获取互斥锁
- 普通互斥量在获取一次互斥锁之后，必须先释放互斥锁才能再次获取互斥锁

递归互斥量比较适合在进行某些递归操作时用来做资源保护。

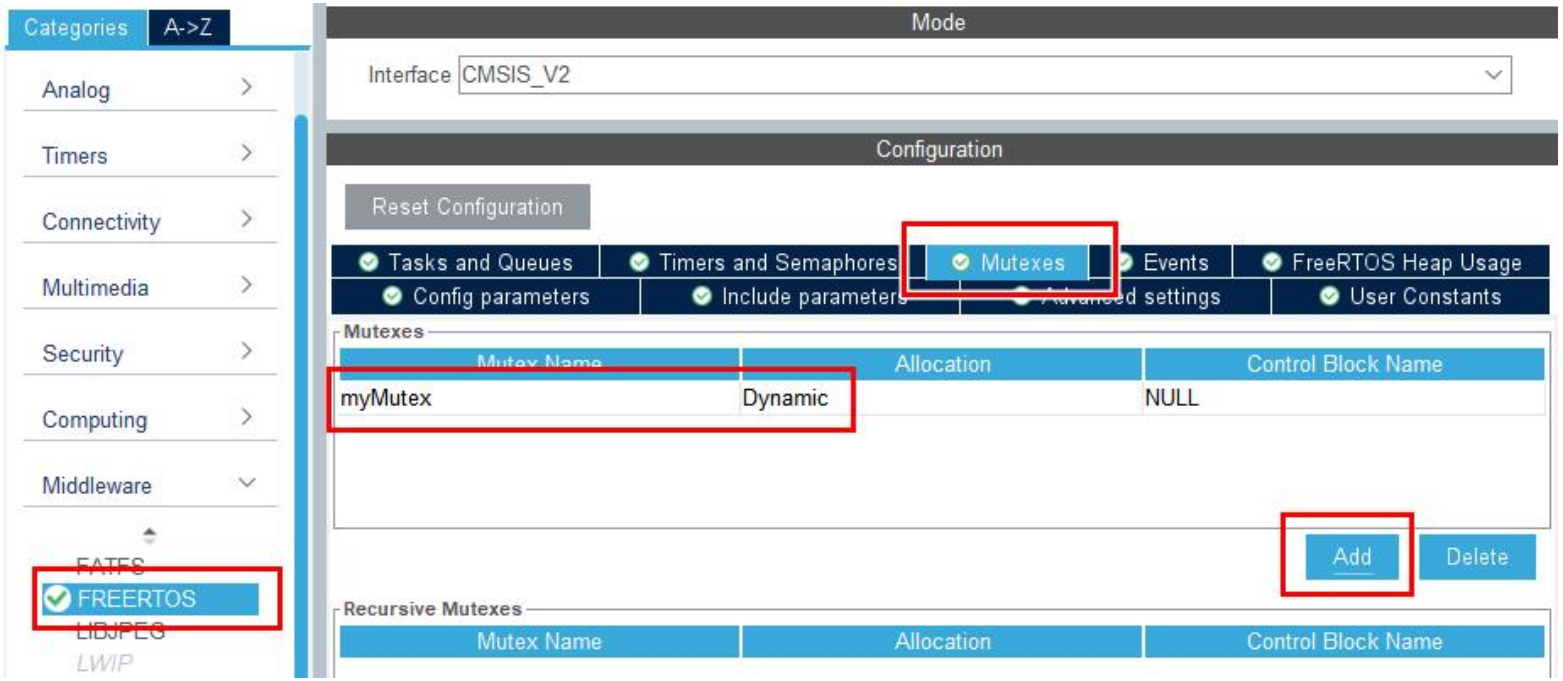
# 多任务系统基本概念——互斥量

FreeRTOS 中关于互斥量的常用函数如下表所示。

| 功能分类 | FreeRTOS原版函数 | CMSIS v2封装函数 | 说明 |
|---|---|---|---|
| 互斥锁管理 | xSemaphoreCreateMute<br>xStaticxSemaphoreCreate<br>Mutex | osMutexNew | 创建并初始化一个互斥锁对象 |
| | xSemaphoreTake | osMutexAcquire | 获取互斥锁，如果它被锁定，则获取互斥锁或超时值 |
| | xSemaphoreGive | osMutexRelease | 释放出osMutexAcquire 获取的互斥锁 |
| | vSemaphoreDelete | osMutexDelete | 删除互斥锁对象 |

# 多任务系统基本概念——互斥量

如下图所示，用户也可以在CubeMX的FreeRTOS 设置页面中给工程先配置好要用到的互斥量。



# RTOS通信示例——CubeMX工程配置

启动CubeMX后，选择STM32F407VE的器件型号开始一个新的CubeMX工程。添加RCC设置和LED、按键、数码管等外设端口，GPIO设置如下所示。

| Pin Name | Sig... | GPIO o... | GPIO mode | GPIO Pull-up/Pull-down | Maximum outp... | User Label |
|---|---|---|---|---|---|---|
| PA8 | n/a | Low | Output Push Pull | No pull-up and no pull-down | Low | DISLK |
| PA11 | n/a | Low | Output Push Pull | No pull-up and no pull-down | Low | SCK |
| PA12 | n/a | Low | Output Push Pull | No pull-up and no pull-down | Low | A3 |
| PA15 | n/a | Low | Output Push Pull | No pull-up and no pull-down | Low | A0 |
| PC8 | n/a | Low | Output Push Pull | No pull-up and no pull-down | Low | SER |
| PC9 | n/a | Low | Output Push Pull | No pull-up and no pull-down | Low | DISEN |
| PC10 | n/a | Low | Output Push Pull | No pull-up and no pull-down | Low | A1 |
| PC11 | n/a | Low | Output Push Pull | No pull-up and no pull-down | Low | A2 |
| PE1 | n/a | n/a | Input mode | Pull-up | n/a | K1 |
| PE2 | n/a | n/a | Input mode | Pull-up | n/a | K2 |
| PE3 | n/a | n/a | Input mode | Pull-up | n/a | K3 |
| PE4 | n/a | n/a | Input mode | Pull-up | n/a | K4 |
| PE5 | n/a | n/a | Input mode | Pull-down | n/a | K5 |
| PE6 | n/a | n/a | Input mode | Pull-down | n/a | K6 |
| PE8 | n/a | High | Output Push Pull | No pull-up and no pull-down | High | L1 |
| PE9 | n/a | High | Output Push Pull | No pull-up and no pull-down | High | L2 |
| PE10 | n/a | High | Output Push Pull | No pull-up and no pull-down | High | L3 |
| PE11 | n/a | High | Output Push Pull | No pull-up and no pull-down | High | L4 |
| PE12 | n/a | High | Output Push Pull | No pull-up and no pull-down | High | L5 |
| PE13 | n/a | High | Output Push Pull | No pull-up and no pull-down | High | L6 |
| PE14 | n/a | High | Output Push Pull | No pull-up and no pull-down | High | L7 |
| PE15 | n/a | High | Output Push Pull | No pull-up and no pull-down | High | L8 |

# RTOS通信示例——CubeMX工程配置

- 展开左侧列表中的“Middleware”中间件栏，选择“FreeRTOS”模块，设置为**CMSIS_V2**版本。
- 选择下方FreeRTOS配置标签栏中的“**Tasks and Queues**”标签，对FreeRTOS的任务和消息队列设置如右所示。
- 继续选择“Timers and Semaphores”标签，对FreeRTOS的信号量设置如右所示。
- 最后选择“Mutexes”标签，添加一个名为“MyMutex”的互斥量，如右所示。

FREERTOS Mode and Configuration

Mode

Interface CMSIS_V2

Tasks and Queues | Timers and Semaphores | Mutexes | Event

Config parameters | Include parameters | Advanced settin

Tasks

| Task Name | Priority | Stack Size (Words) | Entry Function | Code Ge... | Param... |
|---|---|---|---|---|---|
| defaultTask | osPriorityNormal | 128 | StartDefaultTask | Default | NULL |
| keyScanTask | osPriorityNormal | 128 | StartKeyScanTask | Default | NULL |
| segDispTask | osPriorityLow | 128 | StartSegDispTask | Default | NULL |

Queues

| Queue Name | Queue Size | Item Size | Allocation | |
|---|---|---|---|---|
| myQueue01 | 4 | 8 | Dynamic | NULL |

User Constants | Tasks and Queues | Timers and Semaphores

Config parameters | Include parameters | Advanced settings

Binary Semaphores

| Semaphore Name | Allocation | Control Block Name | Initial State |
|---|---|---|---|
| BinSem | Dynamic | NULL | Available |

Counting Semaphores

| Semaphore Name | Max Count | Allocation | Control Block Na... | Initial Count |
|---|---|---|---|---|
| CntSem | 5 | Dynamic | NULL | 0 |

Mutexes | Events | FreeRTOS Heap Usage

User Constants | Tasks and Queues | Timers and Semaphores

Config parameters | Include parameters | Advanced settings

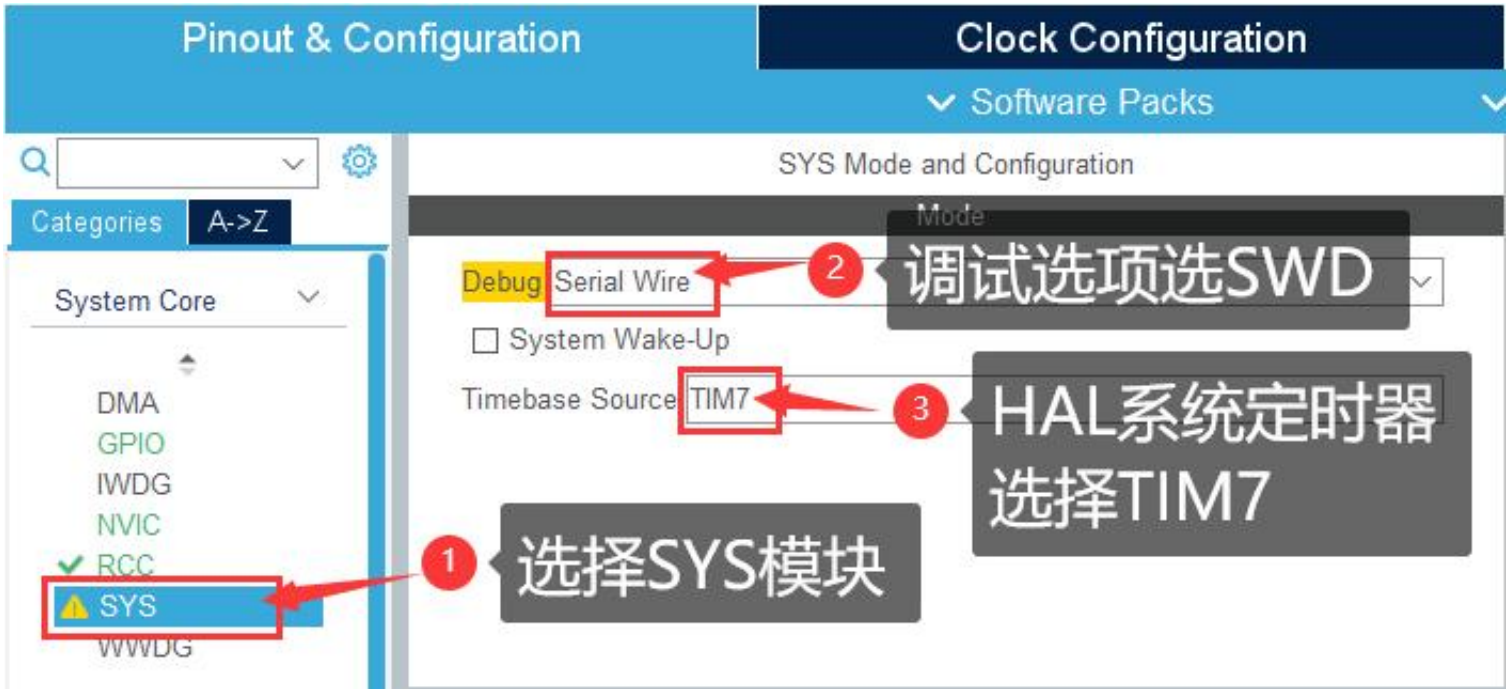
Mutexes

| Mutex Name | Allocation | Control Block Name | State |
|---|---|---|---|
| myMutex | Dynamic | NULL | Available |

# RTOS通信示例——CubeMX工程配置

因为FreeRTOS会自动接管系统滴答定时器中断，因此还需要修改一下左侧“System Core”栏目中的SYS设置：

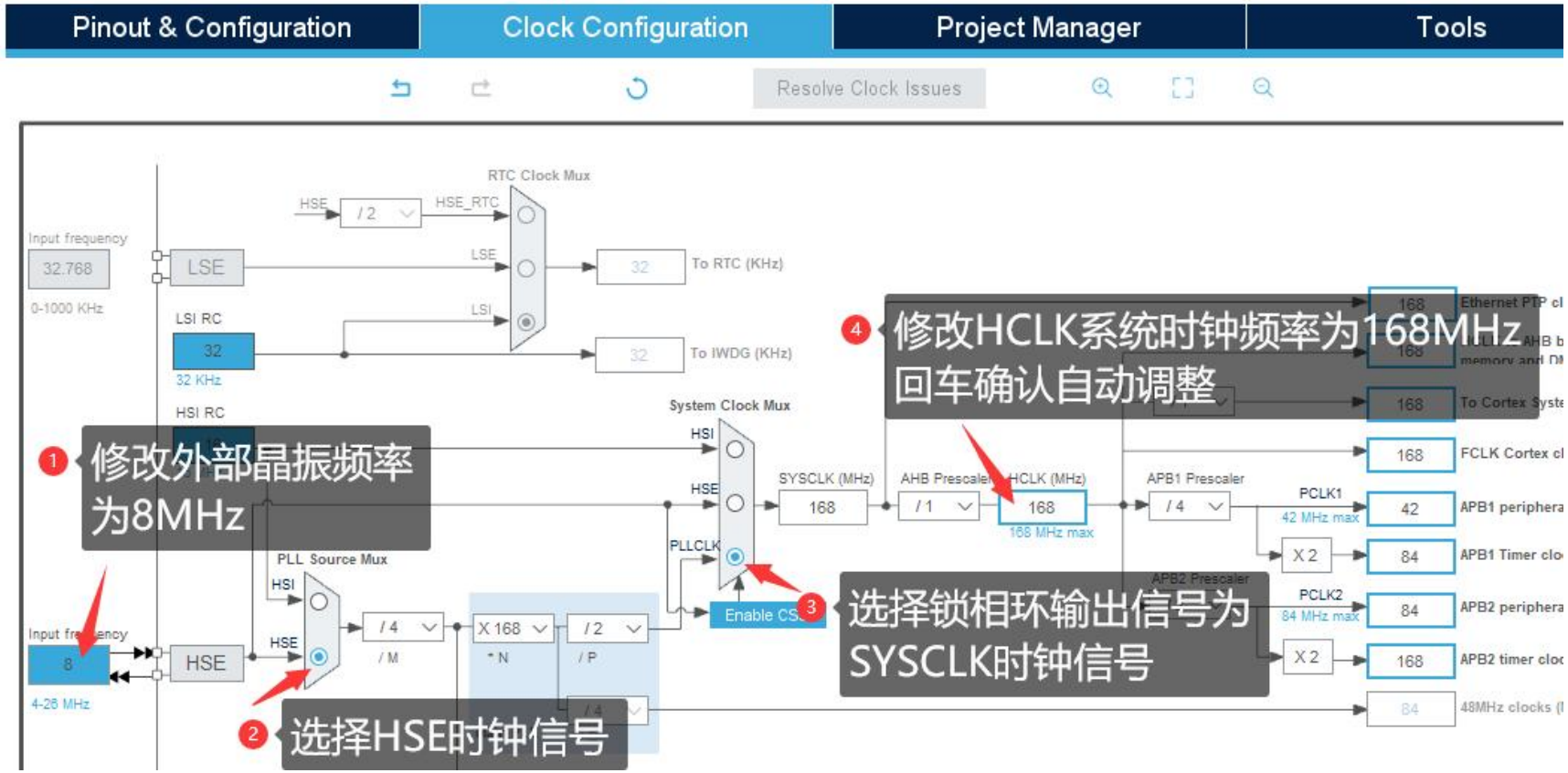
- 将系统HAL定时器的时钟源从SysTick改为其它定时器
- 如下图所示，选择TIM7作为新的时钟源



# RTOS通信示例——CubeMX工程配置

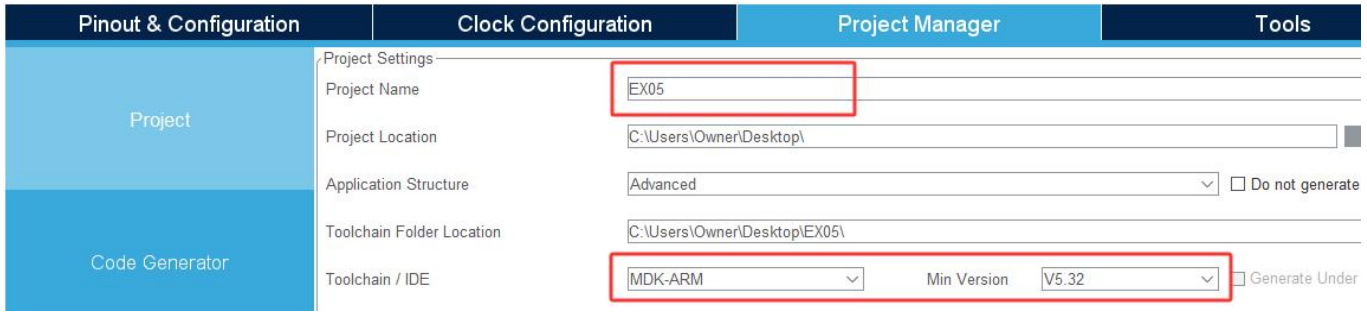
完成“Pinout & Configuration”页面的设置后

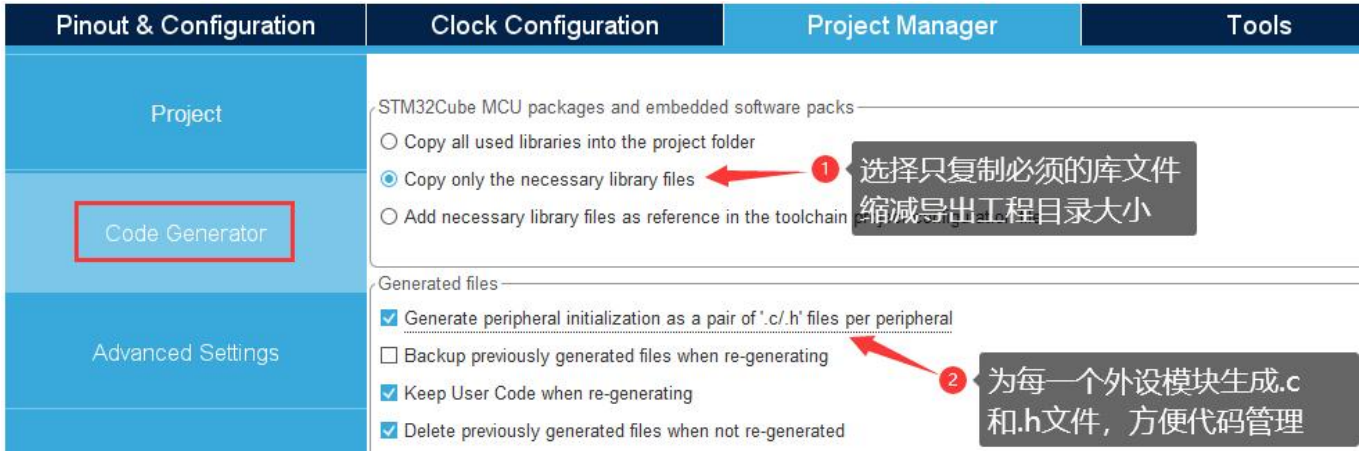
- 单击主界面标签栏中的“Clock Configuration”标签切换到系统时钟配置如下图所示。本示例使用外部时钟源为8MHz晶振，单片机时钟频率设置为168MHz。



# RTOS通信示例——导出MDK工程

- 单击CubeMX界面上方的“Project Manager”按钮切换到输出工程管理页面如右图所示。
- 按照图中设置要生成的STM32工程名为“EX05”，选择工程开发环境为“MDK-ARM V5.32”以上版本。
- 修改库文件复制选项及生成文件选项。最后导出生成MDK工程及源码文件。



# RTOS通信示例1——数码管显示同步与计数控制

在之前导出生成的EX05工程基础上，本次示例将实现通过按键控制数码管显示计数信号量个数的功能。当按键K1、K4按动时计数信号量数值变动，数码管显示数值保持2秒，而后关闭显示。

首先参考实验2中的数码管显示驱动，在gpio.c和gpio.h文件中添加数码管显示函数和按键扫描函数。然后修改freertos.c文件中按键扫描任务代码如下：

```
void StartKeyScanTask(void *argument) {
  /* USER CODE BEGIN StartKeyScanTask */
  for(;;)  {          int key1, key2;
                      if ((key1 = ReadKey()) > 0)     {              // 如果有按键动作
                              osDelay(10);            key2 = ReadKey();   // 延时消抖
                              if (key1 == key2)     {
                                      while (ReadKey() > 0);            // 等待按键放开
                                      if (1 == key1)          osSemaphoreRelease(CntSemHandle);                 // 计数加
                                      else if (4 == key1)   osSemaphoreAcquire(CntSemHandle, 10);           // 计数减
                                      osSemaphoreRelease(BinSemHandle);                                 // 释放信号量
                              }
                      }
    osDelay(1);
  }
  /* USER CODE END StartKeyScanTask */
}
```

# RTOS通信示例1——数码管显示同步与计数控制

继续修改freertos.c文件中数码管显示任务代码如下：

```
void StartSegDispTask(void *argument) {
  /* USER CODE BEGIN StartSegDispTask */
  uint32_t show_tick = 0;  char str[8] = {'0', '0', '0', '0'};
  for(;;)  {
                    if (show_tick == 0 && osSemaphoreAcquire(BinSemHandle, 10) == osOK)        // 获取信号量
                              show_tick = osKernelGetTickCount();                    // 获取成功则刷新显示时间戳
                    if (show_tick > 0 && osKernelGetTickCount() <= show_tick + 2000) {
                              int ncnt = osSemaphoreGetCount(CntSemHandle);
                              str[3] = '0' + ncnt;
                              DispSeg(str);
                    }
                    else if (show_tick > 0) {
                              DispSeg("    ");        show_tick = 0;
                    }
    osDelay(1);
  }
  /* USER CODE END StartSegDispTask */
}
```

以上代码修改之后，编译、下载测试运行，查看当按动K1、K4按键时的数码管显示效果。由此验证信号量在任务同步和任务间通信场景的作用。

# RTOS通信示例2——数码管显示同步与计数控制

接下来，演示互斥量在共享资源访问场景下的作用。首先备份EX05工程，并在此基础上继续修改freertos.c文件中数码管显示任务和默认任务代码如下：

```c
void StartDefaultTask(void *argument) {
  /* USER CODE BEGIN StartDefaultTask */
  uint32_t show_tick = 0;  osDelay(10);
  for(;;)  {
    if (show_tick == 0 &&
      osSemaphoreAcquire(BinSemHandle, 10) == osOK) {
        osMutexAcquire(myMutexHandle, osWaitForever);
        show_tick = osKernelGetTickCount() + 2000;
    }
    if (osKernelGetTickCount() <= show_tick)
        DispSeg("1234");
    else if (show_tick > 0) {
        DispSeg("    ");
        show_tick = 0;
        osMutexRelease(myMutexHandle);
    }
    osDelay(1);
  }
  /* USER CODE END StartDefaultTask */
}
```

```c
void StartSegDispTask(void *argument) {
  /* USER CODE BEGIN StartSegDispTask */
  uint32_t show_tick = 0;  osDelay(10);
  for(;;)  {
    if (show_tick == 0 &&
      osSemaphoreAcquire(BinSemHandle, 10) == osOK) {
        osMutexAcquire(myMutexHandle, osWaitForever);
        show_tick = osKernelGetTickCount() + 2000;
    }
    if (osKernelGetTickCount() <= show_tick)
        DispSeg("5678");
    else if (show_tick > 0) {
        DispSeg("    ");
        show_tick = 0;
        osMutexRelease(myMutexHandle);
    }
    osDelay(1);
  }
  /* USER CODE END StartSegDispTask */
}
```

# RTOS通信示例2——数码管显示同步与计数控制

最后，修改freertos.c文件中按键扫描任务代码如下：

```
void StartKeyScanTask(void *argument) {
  /* USER CODE BEGIN StartKeyScanTask */
  osSemaphoreAcquire(BinSemHandle, 10);       // 获取默认为1的二值信号量，避免上电显示1234
  for(;;)  {
              int key1, key2;
              if ((key1 = ReadKey()) > 0) {
                        osDelay(10);        key2 = ReadKey();    // 按键消抖
                        if (key1 == key2)  {
                                  while (ReadKey() > 0);         // 等待按键放开
                                  osSemaphoreRelease(BinSemHandle);        // 释放二值信号量，开启数码管显示
                        }
              }
    osDelay(1);
  }
  /* USER CODE END StartKeyScanTask */
}
```

修改完成后，编译、下载程序到学习板上，按动一次按键可以观察到数码管显示“1234”，2秒内再按一次按键可以观察到数码管接下来显示“5678”。程序中用了互斥量操作来保护数码管的显示，避免默认任务和数码管显示任务同时操作数码管显示，导致显示异常的问题。大家可以把上页代码中的黄底粗体代码行注释掉，重新编译下载，对比观察没有互斥量时的数码管显示异常情况。

# RTOS通信示例3——按键发送消息调整参数

接下来，演示消息队列在任务间数据通信场景的应用。再次备份EX05工程，并在此基础上继续修改freertos.c文件中按键扫描任务和数码管显示任务代码如下：

```
void StartKeyScanTask(void *argument) {
  /* USER CODE BEGIN StartKeyScanTask */
  for(;;)  {
    int key1, key2;
    if ((key1 = ReadKey()) > 0) {
      osDelay(100);  key2 = ReadKey();
      if (key1 == key2) {
        if (1 == key1) {
          char str[8] = "ADD";
          osMessageQueuePut(myQueue01Handle, str, 0, 0);
        }
        else if (4 == key1) {
          char str[8] = "SUB";
          osMessageQueuePut(myQueue01Handle, str, 0, 0);
        }
      }
    }
    osDelay(1);
  }
  /* USER CODE END StartKeyScanTask */
}
```

```
void StartSegDispTask(void *argument) {
  /* USER CODE BEGIN StartSegDispTask */
  int cnt = 0;
  char dat[8];
  char str[10];
  sprintf(str, "%4d", cnt);
  for(;;)  {
    if (osMessageQueueGet(myQueue01Handle, dat, 0, 0) == osOK) {
      if (dat[0] == 'A' && dat[1] == 'D' && dat[2] == 'D')
        ++cnt;
      else if (dat[0] == 'S' && dat[1] == 'U' && dat[2] == 'B')
        --cnt;
      sprintf(str, "%4d", cnt);
    }
    DispSeg(str);
  }
  /* USER CODE END StartSegDispTask */
}
```

# RTOS通信示例3——按键发送消息调整参数

最后，修改freertos.c文件中默认任务代码如下：

```
void StartDefaultTask(void *argument) {
  /* USER CODE BEGIN StartDefaultTask */
  for(;;)  {
    osDelay(1);
  }
  /* USER CODE END StartDefaultTask */
}
```

修改完成后，编译、下载程序到学习板上，按动一次按键K1或者K4可以观察到数码管显示的参数值会发生增加或减少变化，一直按住K1或K4时可以看到数码管显示参数在连续发生变化。程序中用了消息队列来传递参数控制命令，大家可以尝试再增加K2清空和K3直接设置参数为999的控制命令。注意发送的命令字节数大小要符合消息队列要求的消息大小。

# 实验内容

**1. 示例工程实践练习**

参照本次实验讲义内容，创建并复现EX05示例工程，最后在学习板上测试通过。

**2. 流水灯按键控制练习**

基于本章示例3的EX05工程，在默认任务中实现一个速度、方向和启停可控制的流水灯，通过按键扫描任务向流水灯任务发送不同的控制命令，用K1、K4进行速度调节，K2控制流水灯变化方法，K4控制流水灯启动/暂停，在学习板上进行测试验证。不可使用全局变量和任务通知，不可在流水灯任务中扫描按键。

**3.** 到学习通上的对应课程作业中，完成提交本次实验的**课堂作业**。

# 下一次实验06

## 串口通信实验

请提前做好预习